

Gaussian Component-wise Boosting

Luca Messerschmidt Bjarne Herbst Benedict B. Schnitzler

2026-05-21

Overview

TODO: add citation

This vignette introduces the basic workflow for fitting Gaussian component-wise boosting models using **asp26boost**.

Boosting is a stage-wise optimization procedure that combines many simple model updates into a flexible prediction model. From a statistical perspective, boosting can be viewed as an iterative empirical risk minimization algorithm. Starting from an initial estimate, the model is repeatedly improved by fitting simple base learners to the current residual information and updating only the best-performing model component at each iteration. Early stopping acts as a form of regularization and often leads to sparse models by selecting only a subset of available predictors.

For Gaussian response models, component-wise boosting minimizes the squared-error loss. Beginning with an intercept-only model, each predictor is fitted separately to the current residuals. The predictor yielding the largest reduction in residual sum of squares is selected and updated by a shrunk step determined by the learning rate. Repeating this procedure yields coefficient estimates, variable selection, and regularization simultaneously.

The implementation provided by **asp26boost** follows this component-wise boosting framework and offers methods for model fitting, prediction, summarization, and visualization.

Installation

TBD how to distribute package (TODO)

```
devtools::install_gitlab(  
  repo = "bjarne.herbst01/asp26boost",  
  host = "https://gitlab.gwdg.de"  
)
```

Simulated Sample Data

To illustrate the basic functionality, we generate a synthetic regression dataset with a sparse underlying signal.

```
set.seed(27092001)  
  
n <- 1000 # number of observations  
p <- 10   # number of covariates  
  
X <- matrix(  
  rnorm(n * p),  
  nrow = n,  
  ncol = p  
)
```

```

colnames(X) <- paste0("x", seq_len(p))

beta <- c(2, -1.5, 1, rep(0, p - 3))

y <- X %*% beta + rnorm(n)

sample_data <- data.frame(
  y = as.numeric(y),
  X
)

head(sample_data)
>           y          x1          x2          x3          x4          x5
> 1  1.9923565 -0.4261948 -1.1171738  0.48469532  1.2131425 -0.336170941
> 2 -0.7356261 -0.7131972  0.4245545  0.44002122 -0.8269670 -0.942629273
> 3 -5.4848620 -1.8385995  0.3599087 -0.22333695 -0.4738789 -0.996149133
> 4 -1.9246527  0.8625711  1.5398105 -0.07525713  1.4213003 -0.003867292
> 5 -3.2200384 -1.0810953  1.3918728  0.35499283 -0.5583812  0.978657121
> 6 -4.1853657 -1.7623401 -0.8189664 -1.73188716 -1.5627475 -0.983039525
>           x6          x7          x8          x9          x10
> 1 -1.2355394  0.70395175  2.1222730 -1.7657446  0.4977308
> 2  0.8983855 -1.19335497 -0.9879921  0.2926119  0.8580337
> 3 -2.0669550 -0.65491769  0.5644572  0.5134706  0.3168015
> 4  0.4912211  0.22194407  0.7515952 -1.4363766  0.8185884
> 5 -0.7280939  0.04176014  0.3786812 -0.3892265 -1.1157866
> 6 -0.8300621 -2.22611065  0.9980038  0.3107517 -0.5029159

```

The response depends only on the first three predictors. The remaining covariates contain noise and are included to illustrate the variable-selection behavior of component-wise boosting.

Model Fitting

Gaussian component-wise boosting models are fitted using `boost_gaussian()`.

```

fit <- boost_gaussian(
  x = sample_data[, -1],
  y = sample_data$y,
  mstop = 100,
  nu = 0.05
)

```

The most important tuning parameters are:

- `mstop`: number of boosting iterations.
- `nu`: learning rate controlling the size of each boosting update.

Internally, predictors are standardized before fitting. The model is initialized with the mean response and updated iteratively by selecting the predictor that provides the largest reduction in squared-error loss.

Printing the fitted object provides a compact overview of the model.

```

fit
>
> Gaussian Gradient Boosting
> -----

```

```

> Call:
> boost_gaussian(x = sample_data[, -1], y = sample_data$y, mstop = 100,      nu = 0.05)
>
> Tuning parameters:
> mstop : 100
> nu    : 0.05
>
> Predictors:
> Total           : 10
> Selected (>= 1 time) : 3
> Never selected (coef=0): 7
>
> Training fit:
> RSS           : 1151.704
> MSE           : 1.1517
> R-squared     : 0.8628
>
> Selected variables and selection counts:
>
> x1 x2 x3
> 39 34 27

```

Typical output includes:

- the original model call
- the tuning parameters specified during the model call
- The number of predictors, including which were (not) selected
- basic training-fit statistics
- the names of the selected variables and selection frequencies

Model Summary

A more detailed summary can be obtained using `summary()`.

```

summary(fit)
>
> Gaussian Gradient Boosting - Model Summary
> -----
> Call:
> boost_gaussian(x = sample_data[, -1], y = sample_data$y, mstop = 100,      nu = 0.05)
>
> Dimensions : n = 1000  p = 10 ( 3 selected, 7 zero )
>
> Iterations : mstop = 100  nu = 0.05
>
> Residuals:
>      Min       1Q   Median  Mean       3Q      Max
> -3.4117 -0.7242 -0.0018   0  0.6963  3.4373
>
> RSS : 1151.7041    MSE : 1.1517    R-squared : 0.8628
>
> Coefficients on original predictor scale:
> Variable              Coefficient
> -----

```

```

> (Intercept)          0.0132
> x1                   1.7458
> x2                   -1.252
> x3                   0.7514
>
> Variables set to zero (never selected):
> x4, x5, x6, x7, x8, x9, x10

```

The summary object provides additional information on:

- summary of residuals
- coefficient estimates on the original predictor scale for selected predictors
- list of non-selected predictors (`coefficient = 0`)

Prediction

Predictions can be computed using `predict()`.

```

predict(fit)
> [1]  1.0320482 -1.4328089 -3.8150911 -0.4652747 -3.3500622 -3.3395701
> [7]  1.3212026  1.9802284  0.6068381 -2.6975222 -5.5008491  1.5849156
> [13]  3.5923277 -2.1700128 -3.8036107 -0.5313185 -3.2792155  0.4686502
> [19]  0.8808212  3.0101737 -3.3505316  0.5407987 -0.7285406 -1.6991726
> [25]  1.3659439  0.7597137 -0.8174199  2.7065932 -3.3545771 -3.8732009
> [31]  0.1693680 -2.1134047  0.5979673  1.4359644  0.8240144 -1.2170405
> [37]  2.3368156 -2.9262133  6.2893610  0.9742130  3.0691298  1.3275140
> [43] -0.4203040  0.0762718 -0.1800517 -0.4903056 -0.7351934 -1.0329935
> [49]  1.7752388 -0.7387061 -3.6704423  0.9982863  4.2573478  2.6972791
> [55]  4.2529715 -1.4359229 -0.8590133  1.3743619 -0.4485494  2.4073769
> [61] -6.4854641  2.5259225 -0.4755758 -3.1026599 -2.5607439 -3.3318933
> [67]  3.0449258 -1.9550205 -0.5572617 -0.6920221 -2.1570458  1.1429778
> [73]  0.1946367 -3.3570676 -0.2484705  2.4120557 -0.3592470  1.5232940
> [79] -3.3498072 -5.8454811  2.4273088  0.1208657  1.3004832 -3.5301845
> [85]  0.7521441  1.5992704  2.8454883  5.8820165  0.9128841  0.5150026
> [91] -1.6198088 -2.8762245 -2.5613966  0.4998763 -2.4835711 -1.6942517
> [97]  0.4356116 -3.0802583  0.1679386 -2.1303667
> [ reached 'max' / getOption("max.print") -- omitted 900 entries ]

```

When no additional data are supplied, fitted values for the training sample are returned.

Predictions for new observations can be generated via the `newdata` argument.

```

new_data <- sample_data[1:5, -1]

predict(
  fit,
  newdata = new_data
)
> [1]  1.0320482 -1.4328089 -3.8150911 -0.4652747 -3.3500622

```

The prediction method also supports evaluating models at earlier boosting iterations.

```

predict(
  fit,
  newdata = new_data,
  mstop = 25
)

```

```
)
> [1] 0.1503980 -0.9684167 -2.2483690 0.1483081 -1.8841497
```

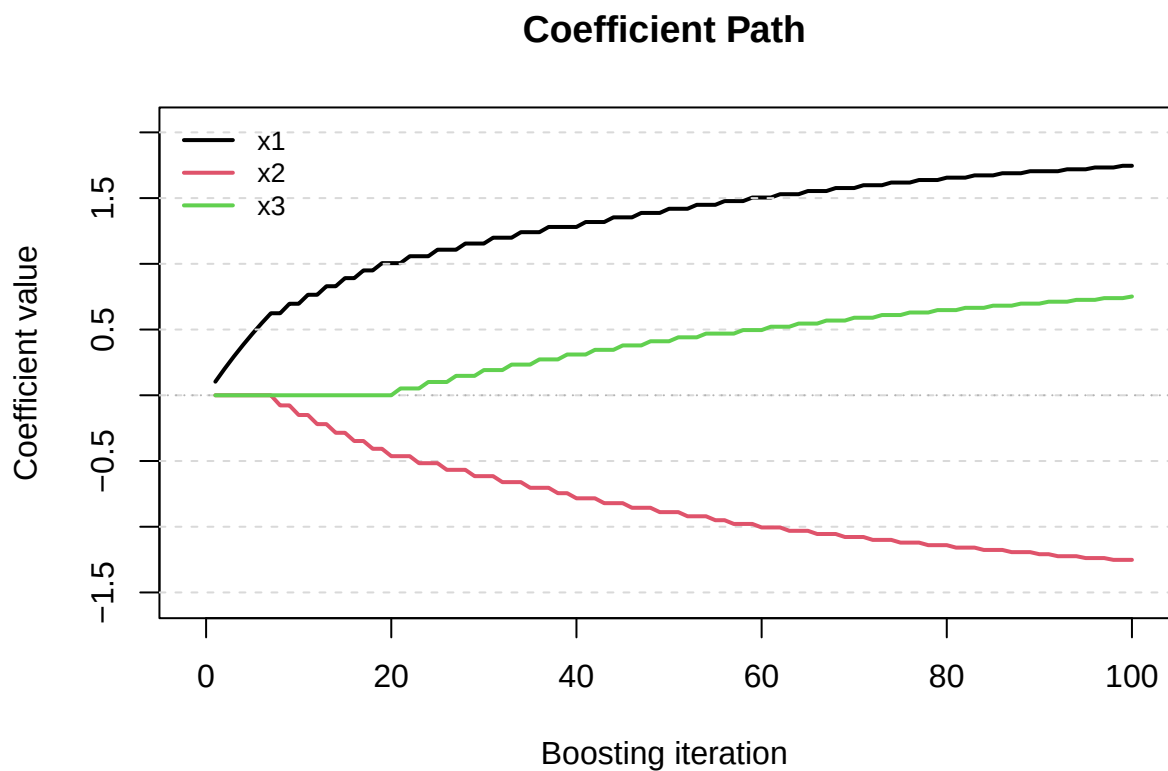
Note For in-sample early-stopped predictions, the model must have been fitted with `keep_path = TRUE`, which is the default when `keep_path` is unspecified.

Model Diagnostics and Visualization

The package provides graphical diagnostics through the `plot()` method.

Coefficient trajectories across boosting iterations can be visualized using `plot()` method by specifying `type = "path"`.

```
plot(
  fit,
  type = "path",
  top_n = 8,
  scale = "original"
)
```



By default, the eight coefficients with the highest selection frequencies are displayed. Coefficients may be shown either on the original (default) predictor scale or on the standardized training scale.

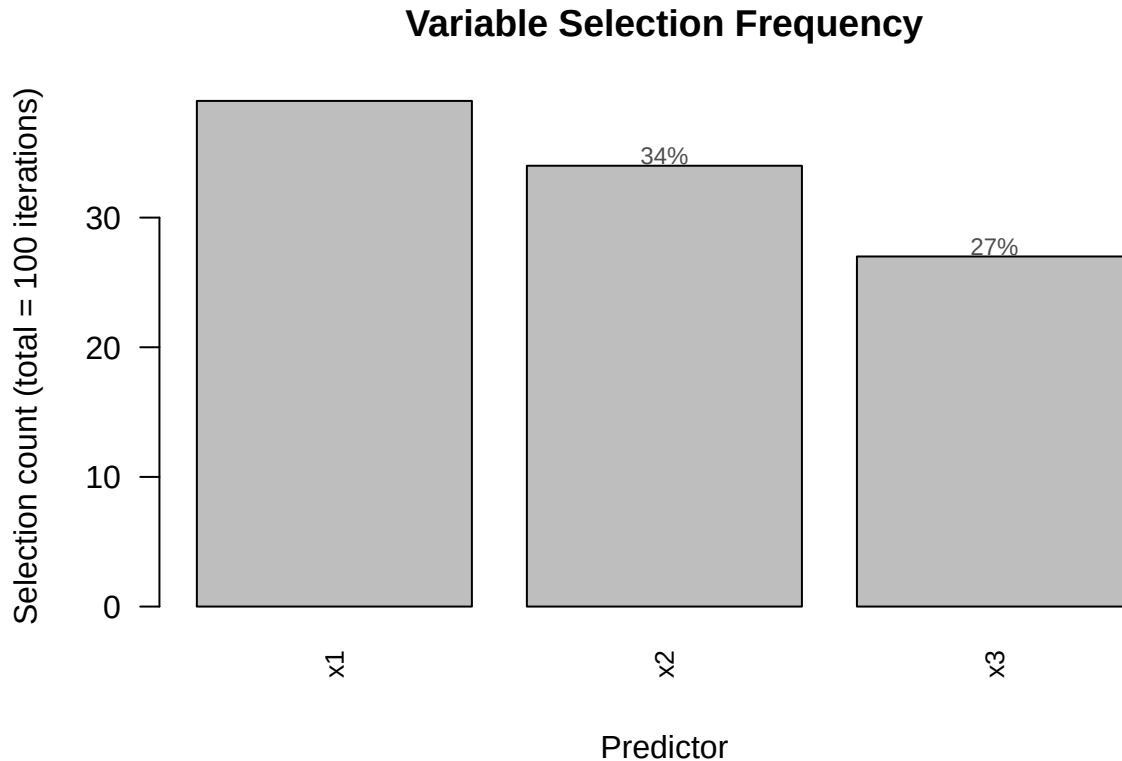
Variable selection frequencies can be inspected by setting `type = "frequency"`.

```
plot(
  fit,
```

```

type = "frequency",
top_n = 8
)

```



This visualization summarizes how often each predictor was selected during the boosting process and provides a simple measure of variable importance. Again the number of Predictors displayed can be adjusted via `top_n`.

Note To use the `plot()` diagnostics the model must be fitted with `keep_path = TRUE`, which is the default.

Cross-Validation for Selecting the Stopping Iteration

A key tuning parameter in component-wise boosting is the number of boosting iterations, denoted by `mstop`. While increasing the number of iterations typically decreases the training error, excessively large values may lead to overfitting. Consequently, early stopping serves as the primary regularization mechanism in statistical boosting, encouraging sparsity in the selected variables.

A common strategy for selecting an appropriate stopping iteration is cross-validation. The data is repeatedly split into training and validation subsets. Models are fitted on the training folds and evaluated on the corresponding validation folds. The value of `mstop` that minimizes the estimated out-of-sample prediction error is then selected.

The function `cv_boost_gaussian()` implements k-fold cross-validation for Gaussian component-wise boosting models.

```

K = 5 # Number of folds during cross-validation

```

```

cv_fit <- cv_boost_gaussian(
  x = sample_data[, -1],

```

```

y = sample_data$y,
k = K,
mstop = 1000,
nu = 0.05,
seed = 27092001
)

```

Inspecting Cross-Validation Results

The resulting object contains the selected variables in the optimal model, the optimal `mstop`, and a final model refitted on the complete dataset using the optimal value of `mstop`.

```

cv_fit
>
> Call:
> cv_boost_gaussian(x = sample_data[, -1], y = sample_data$y, k = K,
>   mstop = 1000, nu = 0.05, seed = 27092001)
>
> Optimal mstop: 333
> Selected variables in new model:
> [1] "x1" "x2" "x3" "x10" "x8" "x6" "x9" "x7"

```

A more detailed summary can be obtained via:

```

summary(cv_fit)
> Model Summary:
> Residuals:
>   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
> -3.7399 -0.6378 -0.0256  0.0000  0.6476  3.0978
>
> Coefficients:
> [1]  0.10370520  0.09851994  0.09359394  0.08891425  0.08446853  0.08024511
> [7]  0.07623285 -0.07472191  0.07233560 -0.07090294  0.06863758 -0.06727915
> [13]  0.06512862 -0.06384057  0.06179904 -0.06057774  0.05863968 -0.05748166
> [19]  0.05564184 -0.05454383  0.05312876  0.05262791 -0.05173960  0.05028829
> [25]  0.04977695 -0.04907975  0.04759976  0.04708015 -0.04655683  0.04505504
> [31]  0.04452919 -0.04416378  0.04264645  0.04211620 -0.04189391  0.04036670
> [37]  0.03983373 -0.03974086  0.03820888 -0.03774178  0.03763148  0.03616660
> [43] -0.03580018  0.03559362  0.03423355 -0.03395861  0.03366591  0.03240387
> [49] -0.03221190  0.03184243  0.03067204 -0.03055516  0.03011754  0.02903282
> [55] -0.02898375  0.02848591 -0.02750193  0.02747259  0.02694254 -0.02608731
> [61]  0.02600487  0.02548264 -0.02474556  0.02461561  0.02410170 -0.02347291
> [67]  0.02330062  0.02279545 -0.02226581  0.02205592  0.02155987 -0.02112087
> [73]  0.02087775  0.02039113 -0.02003489  0.01976256  0.01928563 -0.01900482
> [79]  0.01870697  0.01823994 -0.01802779  0.01770781  0.01725085 -0.01710106
> [85]  0.01676205  0.01631529 -0.01622203  0.01586683  0.01543036 -0.01538826
> [91]  0.01501946 -0.01461411  0.01457660  0.01421743 -0.01386223  0.01378657
> [97]  0.01345825 -0.01314908  0.01303928  0.01273963
> [ reached 'max' / getOption("max.print") -- omitted 233 entries ]

```

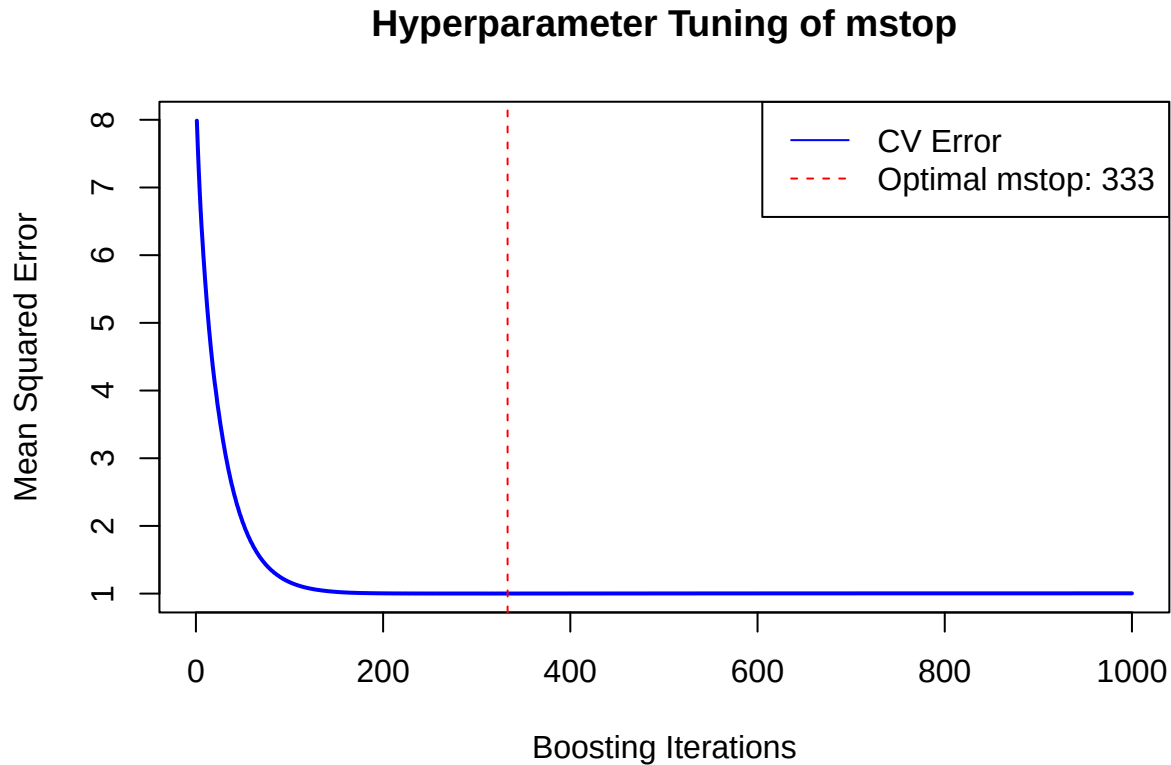
This gives additional information on the Residuals and the Coefficients.

TODO: Coefficients seems to be to many??

Visualizing the Cross-Validation Curve

The estimated validation error can be visualized using the plot method.

```
plot(cv_fit)
```



The blue curve represents the mean validation error across folds as a function of the boosting iteration. The vertical dashed line indicates the selected stopping iteration minimizing the estimated prediction error.

Prediction Using the Cross-Validated Model

Predictions from a cross-validated model can be generated directly via `predict()`.

```
predict(cv_fit)
> [1]  1.41209420 -1.56601095 -4.34525306 -0.55282902 -3.86830555 -3.87621297
> [7]  1.72516953  2.03873312  0.76038983 -3.09200977 -6.46107608  1.84108096
> [13]  4.24452061 -2.53509019 -4.25466182 -0.62918828 -3.62179145  0.58606579
> [19]  1.03423932  3.59558823 -3.97787807  0.58196346 -0.92600752 -1.91568443
> [25]  1.68278960  1.07952187 -0.88060463  2.98589971 -3.87270343 -4.54839288
> [31]  0.29795777 -2.13726144  0.90876244  1.73307284  1.06374018 -1.61932082
> [37]  2.71502883 -3.36885699  7.41060524  1.07169887  3.47073885  1.53779002
> [43] -0.30824893 -0.08946057 -0.02464402 -0.75462901 -0.84739441 -1.17192464
> [49]  2.16729761 -0.68907673 -4.38970131  1.19009519  4.90116887  3.25791868
> [55]  5.13964689 -1.76763533 -0.90014901  1.49691694 -0.57591647  2.90022239
> [61] -7.46229779  3.20627193 -0.46415085 -3.50954800 -2.93015129 -3.90880134
> [67]  3.57180238 -2.18371471 -0.54666451 -0.91865101 -2.37194809  1.03532256
> [73]  0.01670104 -3.73525330 -0.49733757  2.62792329 -0.46871891  1.78854456
```

```
> [79] -3.71025756 -6.86295133  3.00414238  0.05718431  1.29113672 -4.32041697
> [85]  0.87235268  1.71660893  3.01855612  6.79880308  0.93425393  0.44127912
> [91] -2.06367273 -3.42976311 -2.82718702  0.64987564 -2.71595833 -2.22034423
> [97]  0.64449268 -3.64259780  0.12574593 -2.25301529
> [ reached 'max' /getOption("max.print") -- omitted 900 entries ]
```

When no new data is supplied, fitted values from the refitted model are returned.

Predictions for new observations can be obtained by specifying `newdata`.

```
new_data <- sample_data[1:5, -1]

predict(
  cv_fit,
  newdata = new_data
)
> [1]  1.412094 -1.566011 -4.345253 -0.552829 -3.868306
```

Internally, predictions are delegated to the final boosting model fitted using the optimal `mstop` value.

Hyperparameter Grid Search

In some situations, both the stopping iteration `mstop` and the learning rate `nu` may be treated as tuning parameters. The function `cv_boost_grid()` evaluates combinations of candidate values using repeated cross-validation.

```
grid_fit <- cv_boost_grid(
  x = sample_data[, -1],
  y = sample_data$y,
  k = 5,
  mstop_grid = c(50, 100, 200),
  nu_grid = c(0.01, 0.1, 0.3),
  seed = 27092001
)
```

TODO `cv_boost_grid()` funktioniert nicht

The returned object contains:

TODO

Concluding Remarks

This vignette introduced the basic workflow for Gaussian component-wise boosting, including model fitting, prediction, visualization, and cross-validation-based hyperparameter tuning.